

Diverty Park

Informe Técnico Completo
Aplicaciones Web II — Sistemas

TP1 · TP2 · TP3 · TP4

Equipo: Joaquín Barberan · Lisandro · Lucas
Docente: Prof. Andrés Senn
Fecha de entrega: 29/06/2026

Sistema web de gestión para salones de fiestas infantiles · Node.js · Express · PostgreSQL · JWT · bcrypt

TP 1 — Propuesta del Proyecto

1.1 Equipo y Roles

Integrante	Rol principal
Joaquín Barberan	Líder técnico, desarrollo backend y frontend, integración BD
Lisandro	Desarrollo frontend, estilos CSS, testing manual
Lucas	Documentación, testing, apoyo en vistas HTML
IA (Gemini)	Copiloto de código: sugerencias, refactoring y documentación

La metodología aplicada fue **Pair Programming asistida por IA**: los integrantes humanos definieron los requerimientos y testearon localmente, mientras la IA actuó como copiloto de implementación.

1.2 Descripción y Justificación

¿Qué es Diverty Park? Un sistema web de gestión para un salón de fiestas infantiles. Permite al público ver las salas disponibles y realizar consultas de reserva, mientras el equipo administrador puede gestionar el catálogo completo desde un panel web.

¿Qué problema resuelve? La gestión manual (papel o planillas) de salas genera errores, pérdida de información y demoras. Diverty Park centraliza el catálogo en una base de datos con acceso web, con alta/baja/modificación en tiempo real y control de acceso por usuario autenticado.

Propuesta de valor: Un sistema simple, seguro y escalable, construido con tecnologías modernas (Node.js, PostgreSQL, JWT) que puede desplegarse en la nube con mínima configuración.

1.3 Cronograma de Avances

ID	Tarea	Fecha	Estado
T01-03	Setup entorno, base de datos, arquitectura MVC inicial	29/05/2026	✓ Completo
T04-05	Panel de administración frontend y enrutado de estáticos	29/05/2026	✓ Completo
T06-07	Multer para imágenes, reestructuración de carpetas	15/06/2026	✓ Completo
T08-09	Versionado API /api/v1, implementación de Multer en rutas	15/06/2026	✓ Completo
T10	Informes técnicos TP2/TP3	15/06/2026	✓ Completo
T11-12	Tabla usuarios, hashing bcrypt	25/06/2026	✓ Completo
T13-14	JWT, cookies httpOnly, middleware de autenticación, CORS	25/06/2026	✓ Completo
T15-16	Login.html, variables de entorno, informe TP4	25/06/2026	✓ Completo

1.4 Control de Versiones

Se utilizó **Git** con repositorio en **GitHub** (github.com/joaquinbarberan/Final-aw). Cada etapa del proyecto fue registrada con commits atómicos y descriptivos. El archivo `.gitignore` protege las

credenciales excluyendo .env y node_modules/ del repositorio.

TP 2 — Back-end para servir el sitio web

2.1 Stack tecnológico

Tecnología	Versión	Uso
Node.js	v22+	Entorno de ejecución del servidor
Express	5.2.1	Framework web para definir rutas y middlewares
PostgreSQL	18	Base de datos relacional
pg (node-postgres)	8.21	Cliente PostgreSQL para Node.js
dotenv	17.4	Carga de variables de entorno desde .env
multer	2.2	Middleware para subida de archivos
nanoid	5.1	Generación de nombres únicos para imágenes

2.2 Organización del proyecto

El proyecto está organizado bajo el patrón **MVC** con separación clara de responsabilidades:

```
Final-aw/
├── src/
│   ├── controladores/ ← lógica de negocio
│   ├── modelos/       ← consultas SQL
│   ├── routers/       ← definición de endpoints
│   ├── middleware/    ← autenticación JWT
│   ├── app.mjs         ← servidor principal
│   ├── conexion.bd.mjs ← pool PostgreSQL
│   ├── seed.mjs        ← seed de salas
│   ├── seed.usuarios.mjs ← seed de usuarios
│   └── public/         ← frontend estático
│       ├── index.html / salas.html / reservas.html / admin.html / login.html
│       ├── css/ · js/ · imagenes/
│       ├── .env        ← variables de entorno (no en repo)
│       └── .env.example ← plantilla pública de configuración
```

2.3 Comando para recrear la base de datos

Se agregó el script **seed** en `package.json` para recrear toda la base de datos con un solo comando:

```
npm run seed
```

Este comando ejecuta en secuencia:

1. `node src/seed.mjs` — crea la tabla `salas` e inserta los datos iniciales.
2. `node src/seed.usuarios.mjs` — crea la tabla `usuarios` y genera el usuario administrador con contraseña hasheada.

TP 3 — CRUD para administrar el Front-End

3.1 Arquitectura MVC

Capa	Archivo	Responsabilidad
Vista	public/admin.html + admin.js	Interfaz de administración con modal CRUD
Router	src/routers/salas.routers.mjs	Define endpoints, aplica Multer
Controlador	src/controladores/salas.controller.mjs	Valida datos, procesa imágenes, responde
Modelo	src/modelos/salas.model.mjs	Ejecuta queries SQL (SELECT/INSERT/UPDATE/DELETE)
BD	PostgreSQL — tabla salas	Persistencia relacional

3.2 Endpoints CRUD (5 rutas requeridas)

Método	Endpoint	Operación	Código HTTP
GET	/api/v1/salas	Listar todo	200 OK
GET	/api/v1/salas/:id	Leer uno	200 OK
POST	/api/v1/salas	Alta	201 Created
PUT	/api/v1/salas/:id	Modificación	200 OK
DELETE	/api/v1/salas/:id	Baja	200 OK

Adicionalmente se exponen 2 endpoints **públicos de solo lectura** en /salas y /salas/:id para la web del cliente.

3.3 Interfaz de Administración

El panel **admin.html** incluye:

- Tabla con todas las salas cargadas dinámicamente desde la API.
- Modal con formulario para **crear** y **editar** salas, con vista previa de imagen.
- Botón de **eliminar** con confirmación antes de ejecutar el DELETE.
- Mensajes de **éxito** (*alert*) al crear, editar o eliminar.
- Mensajes de **error** si el servidor devuelve un código 4xx/5xx.
- Estadísticas en tiempo real: total de salas, capacidad niños, capacidad adultos.
- Verificación de sesión al cargar: redirige a login.html si no hay JWT válido.

3.4 Esquema SQL de la tabla salas

```
CREATE TABLE IF NOT EXISTS salas (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(255) NOT NULL,  
  capacidad_ninos INT NOT NULL DEFAULT 0,  
  capacidad_adultos INT NOT NULL DEFAULT 0,  
  precio DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
```

```
descripcion TEXT,  
imagen VARCHAR(255),  
alt_imagen VARCHAR(255)  
);
```

TP 4 — Autenticación, Accesos y Seguridad

4.1 Tabla de Usuarios en Base de Datos

```
CREATE TABLE IF NOT EXISTS usuarios (  
  id SERIAL PRIMARY KEY,  
  nombre_usuario VARCHAR(100) NOT NULL UNIQUE,  
  contrasena_hash VARCHAR(255) NOT NULL,  
  creado_en TIMESTAMP DEFAULT NOW()  
);
```

La columna `contrasena_hash` **nunca almacena contraseñas en texto plano**. Solo guarda el hash irreversible generado por `bcrypt`. El usuario administrador inicial se crea ejecutando `npm run seed`.

4.2 Hashing de Contraseñas con bcrypt

Se usa **bcryptjs** con `saltRounds = 10`. El proceso es **irreversible**: aunque alguien acceda directamente a la base de datos, no puede recuperar las contraseñas.

```
// Registrar: genera un hash a partir de la contraseña  
const hash = await bcrypt.hash(contrasena, 10);  
  
// Login: compara contraseña ingresada vs hash guardado  
const esCorrecta = await bcrypt.compare(contrasena, hash);
```

4.3 Autenticación con JWT y Cookies `httpOnly`

1. Usuario ingresa usuario y contraseña en **login.html**.
2. POST a `/api/v1/usuarios/login`: el servidor verifica el hash con `bcrypt`.
3. Si es correcto, genera un **Token JWT** firmado con `JWT_SECRETO` (expira en 8h).
4. El token se guarda en una **cookie httpOnly** → inaccesible desde JS del navegador (protección XSS).
5. Flag `sameSite: 'strict'` → protección adicional contra CSRF.
6. Cada petición al CRUD envía la cookie automáticamente. El middleware **verificarToken** la valida.
7. Token inválido o expirado → servidor responde **401 Acceso Denegado**.
8. Logout: POST a `/api/v1/usuarios/logout` → borra la cookie.

Método	Endpoint	Descripción
POST	<code>/api/v1/usuarios/login</code>	Valida credenciales y setea cookie con JWT
POST	<code>/api/v1/usuarios/logout</code>	Borra la cookie (cierra sesión)
GET	<code>/api/v1/usuarios/verificar</code>	Confirma si la sesión está activa (usa middleware)

4.4 CORS — Justificación

Frontend y backend corren en el **mismo servidor Express** (mismo origen `http://localhost:3000`), por lo que CORS no es estrictamente necesario. Se configuró de forma **restrictiva y preventiva** para el despliegue en producción (ej: frontend en Vercel + backend en Render), donde los dominios serían

distintos. Solo se permite el origen definido en `FRONTEND_URL`. El flag `credentials: true` es obligatorio para que las cookies `httpOnly` viajen entre dominios.

4.5 Variables de Entorno

Variable	Descripción
PORT	Puerto del servidor (default: 3000)
DB_USER / DB_PASSWORD	Credenciales de PostgreSQL
DB_HOST / DB_PORT / DB_DATABASE	Conexión a la base de datos
JWT_SECRETO	Clave para firmar/verificar tokens (nunca subir al repo)
FRONTEND_URL	Origen permitido por CORS en producción

El archivo `.env` está en `.gitignore` y nunca se sube al repositorio. El archivo `.env.example` sirve como plantilla pública con los nombres de las variables sin sus valores reales.